

SenseTrade AI Project Report

1. Project Title

****SenseTrade AI**:** A Hybrid AI Financial Intelligence Platform for Stock Movement Prediction

2. Objective

The objective of this project is to build a production-oriented full-stack platform that predicts stock market movement by combining:

- LSTM for time-series price learning (OHLCV data)
- FinBERT for financial news sentiment analysis

The system generates Buy/Sell/Hold signals with confidence and forecasted price, while also showing explainability through sentiment-driving headlines.

3. Tech Stack

- Frontend: React.js, Tailwind CSS, Recharts
- Backend: Django, Django REST Framework (DRF), JWT Auth
- Database: PostgreSQL
- AI/ML: TensorFlow/Keras (LSTM), Hugging Face Transformers (FinBERT)
- Data Sources: yfinance, NewsAPI, Kaggle CSV datasets (S&P; 500)

4. System Architecture

1. Data Ingestion Layer

- Management command fetches price data and news.
- News headlines are scored using FinBERT.
- Daily aggregated sentiment is stored.

2. Data Storage Layer

- `Ticker`, `PriceHistory`, `NewsHeadline`, `SentimentLog`, `DailySentiment`
- User models: `WatchlistItem`, `PortfolioPosition`
- Prediction logs: `PredictionLog`

3. Hybrid AI Layer

- Builds feature windows using normalized close prices + daily sentiment scores.
- LSTM model predicts next-close movement.
- Output mapped to Buy/Sell/Hold with confidence.

4. API Layer

- JWT register/login endpoints
- Market and prediction endpoints
- Watchlist and portfolio endpoints

5. Frontend Dashboard

- Trading-style chart with sentiment heat overlays
- Prediction card (signal, confidence, current/forecast price)
- Explainable AI panel (top sentiment headlines)

5. Key Features Implemented

- JWT-based authentication flow
- Personalized watchlist and portfolio structure
- Ingestion command for market/news data
- Hybrid prediction API at `/api/predict/`
- Explainability output with headline-level sentiment
- React dashboard with market visualization and prediction UI

6. Kaggle Integration

Used datasets:

- `sp500_companies.csv` for ticker metadata
- `sp500_stocks.csv` for bulk historical OHLCV import
- `sp500_index.csv` kept optional for future market-regime features

A fast import command was added to bulk-load S&P500; data for better model training coverage.

7. Issues Faced and Fixes

- TensorFlow import crash during migrations
- Fixed via lazy imports in AI service layer.
- `NaN` decimal errors from CSV import
- Fixed by robust null/finite checks before DB insert.
- News `description` null integrity error
- Fixed by coercing nullable API values to safe defaults (`""`).
- 401 Unauthorized on market/predict APIs
- Resolved by obtaining JWT and storing access token in frontend localStorage.

8. Current Working Status

- Data ingestion completes successfully for ticker input (example: AAPL).
- Backend and frontend run successfully when services are started.
- Prediction and market endpoints are functional after JWT authentication.
- Bulk historical data import is available for stronger training depth.

9. Signal Logic (Buy/Hold/Sell)

Prediction action is decided using forecasted next price vs current price:

1. Compute percentage change:

`delta = (forecast_price - current_price) / current_price`

2. Signal mapping:

- ``BUY`` if ``delta > 0.01`` (greater than +1%)
- ``SELL`` if ``delta < -0.01`` (less than -1%)
- ``HOLD`` otherwise (between -1% and +1%)

3. Confidence:

- Confidence increases with absolute move size (``abs(delta)``), and is bounded between ``0.50`` and ``0.95``.

10. Live Market Readiness

Current system supports near-live analysis, not full live-trading production:

- Works now:
- Recent data pull and signal generation
- Manual/triggered ingestion updates
- Not yet implemented:
- Real-time streaming and continuous scheduling
- Broker execution integration
- Production-grade risk controls, monitoring, and drift handling

11. How to Run the Project

1. Start backend:

- ``python manage.py runserver 127.0.0.1:8000``

2. Start frontend:

- ``npm run dev``

3. Register/login through auth endpoints

4. Store JWT access token in browser localStorage key: ``sensetrade.access``

5. Use dashboard and prediction endpoint for ticker analysis

12. Future Improvements

- Move model training out of request path to scheduled/offline pipeline
- Persist model artifacts and scaler for faster inference
- Add Celery/Redis for background ingestion and retraining
- Add test suite for API, ingestion, and model logic
- Expand explainability with feature contribution summaries

13. Conclusion

SenseTrade AI demonstrates a practical hybrid-AI architecture that combines market time-series learning and financial sentiment intelligence. The platform is in a functional, extensible state with production-style foundations across backend, frontend, data ingestion, and predictive analytics.